

TP n°5 : Ridge, Lasso

Objectifs pédagogiques :

1. Mettre en place une validation croisée k-fold correctement (pipeline).
2. Comparer la régression des moindres carrés ordinaires, la régression Ridge et la régression Lasso en prédiction.
3. Étudier l'effet de λ : biais-variance, *shrinkage*, sélection de variables.
4. Choisir un modèle via la validation croisée et interpréter les coefficients.

0) Imports

```
import numpy as np
import pandas as pd

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split, KFold
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import GridSearchCV
```

1) Données

On utilise le jeu de données : **California Housing**.

```
X, y = fetch_california_housing(return_X_y=True, as_frame=True)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=0
)

X_train.shape, X_test.shape
```

Question 1. Que représente la variable y ? Quelle est son unité ?

2) Première méthode : la régression des moindres carrés ordinaires

```
ols = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LinearRegression())
])

ols.fit(X_train, y_train)
pred = ols.predict(X_test)

rmse_ols = np.sqrt(mean_squared_error(y_test, pred))
rmse_ols
```

Question 2. Pourquoi met-on un *StandardScaler()* même avec une régression OLS ?

3) Deuxième méthode : la régression ridge et la validation croisée

3.1 Pipeline + grille du paramètre λ

On note dans scikit-learn $\lambda = \alpha$.

```
ridge = Pipeline([
    ("scaler", StandardScaler()),
    ("model", Ridge())
])

alphas = np.logspace(-4, 4, 60)

cv = KFold(n_splits=10, shuffle=True, random_state=0)

grid_ridge = GridSearchCV(
    estimator=ridge,
    param_grid={"model__alpha": alphas},
    scoring="neg_root_mean_squared_error",
    cv=cv,
    n_jobs=-1
)

grid_ridge.fit(X_train, y_train)

grid_ridge.best_params_, -grid_ridge.best_score_
```

Question 3. Pourquoi le `scoring` est négatif ?

3.2 Performance test

```
best_ridge = grid_ridge.best_estimator_
pred = best_ridge.predict(X_test)

rmse_ridge = np.sqrt(mean_squared_error(y_test, pred))
rmse_ridge
```

4) Troisième méthode : la régression Lasso et la validation croisée

Important : la régression Lasso est plus sensible au *scaling* et peut nécessiter plus d'itérations.

```
lasso = Pipeline([
    ("scaler", StandardScaler()),
    ("model", Lasso(max_iter=20000))
])

grid_lasso = GridSearchCV(
    estimator=lasso,
    param_grid={"model__alpha": alphas},
    scoring="neg_root_mean_squared_error",
    cv=cv,
    n_jobs=-1
)

grid_lasso.fit(X_train, y_train)

grid_lasso.best_params_, -grid_lasso.best_score_
```

4.2 Performance test

```
best_lasso = grid_lasso.best_estimator_  
pred = best_lasso.predict(X_test)  
  
rmse_lasso = np.sqrt(mean_squared_error(y_test, pred))  
rmse_lasso
```

5) Comparaison finale

```
pd.DataFrame({  
    "Model": ["OLS", "Ridge (CV)", "Lasso (CV)"],  
    "RMSE_test": [rmse_ols, rmse_ridge, rmse_lasso],  
})
```

Question 4. Quel modèle est meilleur pour le critère RMSE ? Est-ce surprenant ?

Question 5. Comparez les coefficients estimés par les modèles **OLS**, **Ridge** et **Lasso**.

- Les coefficients deviennent-ils plus petits avec la régularisation ?
- Certaines variables disparaissent-elles dans le modèle Lasso ?

6) Interprétation : coefficients et shrinkage

6.1 Extraire les coefficients des trois modèles de régression

```
coef_ols = pd.Series(ols.named_steps["model"].coef_, index=X.columns)  
coef_ridge = pd.Series(best_ridge.named_steps["model"].coef_,  
index=X.columns)  
coef_lasso = pd.Series(best_lasso.named_steps["model"].coef_,  
index=X.columns)  
  
coefs = pd.DataFrame({  
    "OLS": coef_ols,  
    "Ridge": coef_ridge,  
    "Lasso": coef_lasso  
})  
  
coefs
```

6.2 Nombre de coefficients nuls (Lasso)

```
n_zero = (coef_lasso.abs() < 1e-10).sum()  
n_zero
```

Question 6. Quelles variables sont mises à zéro ? Est-ce cohérent ?

7) Courbe RMSE versus alpha

```
ridge_results = pd.DataFrame(grid_ridge.cv_results_)  
lasso_results = pd.DataFrame(grid_lasso.cv_results_)  
  
ridge_curve = ridge_results[["param_model__alpha",  
"mean_test_score"]].copy()  
ridge_curve["rmse_cv"] = -ridge_curve["mean_test_score"]  
ridge_curve = ridge_curve.drop(columns="mean_test_score")
```

```
lasso_curve = lasso_results[["param_model__alpha",
"mean_test_score"]].copy()
lasso_curve["rmse_cv"] = -lasso_curve["mean_test_score"]
lasso_curve = lasso_curve.drop(columns="mean_test_score")

ridge_curve.head()
```

Question 7. Pourquoi la performance se dégrade pour le paramètre alpha très grand ?

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))
plt.semilogx(ridge_curve["param_model__alpha"], ridge_curve["rmse_cv"],
label="Ridge")
plt.semilogx(lasso_curve["param_model__alpha"], lasso_curve["rmse_cv"],
label="Lasso")

plt.xlabel("alpha")
plt.ylabel("RMSE (CV)")
plt.title("Erreur de validation croisée en fonction de alpha")
plt.legend()
plt.show()
```

Questions de synthèse : à rendre

1. Expliquer le rôle de la validation croisée dans le choix de `alpha`.
2. Comparer la régression Ridge et la régression Lasso en termes de :
 - biais-variance,
 - sélection de variables,
 - stabilité quand les variables explicatives sont corrélées.
3. Donner un exemple où vous préféreriez la régression Ridge à la régression Lasso (et inversement).
4. Que se passe-t-il si on retire le `StandardScaler()` ?
5. Pourquoi la régression Ridge est souvent plus stable que la régression Lasso lorsque les variables explicatives sont fortement corrélées ?
6. Si deux variables explicatives sont fortement corrélées, que fait généralement la régression Lasso ? En quoi ce comportement est-il différent de celui de la régression Ridge ?